

Structural Aliasing Analysis in Time-Series Forecasting Using Chronos

Matteo Boniotti, Gianluca Brignoli and Federico Sabbadini*

August 19, 2026

Executive Summary

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam nonummy eiusmod tempor incididunt ut labore et dolore magna aliquam erat voluptat. Ut enim ad minim veniam, quis nostrud exercitation ullamcorper suscipit laborios nisi ut aliquip ex ea commodo consequat. Duis autem vel eum irure reprehenderit in voluptate velit esse nihil molestiae consequat, vel illum dolore eu fugiat nulla pariatur. At vero eos et accusam et iusto odio dignissim ducim qui blandit praesent luptatum delenit atque duos dolor et molestias exceptur sint occaecat cupiditat non provident, simil sunt in culpa qui officia deserunt mollit anim id est laborum et dolor fuga. Et harum dererud facilis est er expedit distinct. Nam liber tempor cum soluta nobis est eligend optio comgue nihil impedit doming id quod maxim placeat facer possim omnis voluptas assumenda est, omnis dolor repellend. Temporibud autem quinusd et aur office debit aut rerum necessitatib saepe eveniet ut er repudiand sint et molestia non recusand. Itaque earud rerum hic tenetur sapiente delectus ut aut preferendis doloribus asperiores repellat. Hanc ego cum tene sententiam, quid est cur verear ne ad eam non possing accommodare torquat nostros quos tu paulo ante cum memoriter tum etiam ergat. Nos amice et nebevol molestias access potest fieri ad augendas cum conscientia factorum poenarum legum odioque civiuda. Et tamen in busdam neque pecuniam modum est neque nonor imperi neque libidinis genepularum reliquarum cupiditat, quas nulla praevident onum dant. Improbabilis parvum minuit, potius inflammatur ut coercenda magister and dedocendesse videantur. Invitat igitur vera ratio bene sanos ad iustitiam, aequitatem fidem. Neque hominum infantum aut iniuste factum conditum qui neque facile efficeret possit duo consuetudines notineri si effecerit, et opes vel fortunam vel ingenium liberalitatem magis conveniunt, dum tamen benevolentibus conciliant et, aptissimum est ad quietem. Endium caritatem praesertim cum omnino nullum sit causam peccandi quatenus inveniunt cupiditatem a natura proficisci facile expleant sine ulla iniuria autem inane sunt ista paranda non est nihil enim desiderabile. Concupis plusque in ipsinuria detrimentum est quam in his rebus emolumentum pariunt iniuri. Itaque ne iustitiam dem rectis quis dixerit per se ipsam optabilem, sed quia cunctis vel pluribus afferat. Nam diligere et carum esse iucundum est propter tutiorem vitam et luptat plenior efficit. Tamen non ob ea solum incommoda quae egenum improbi fugiendae improbitate putamus sed multum etiam magnum quod cuius.

Time-Series Forecasting

Time-series forecasting is a cornerstone of modern predictive analytics, and the instrument through which organisations anticipate the dynamic, state-dependent behaviour of complex systems. Whether the task is predicting fluctuating loads in renewable energy grids, identifying macro-economic trends, or monitoring structural health in industrial machinery, projecting historical sequential data into future horizons shifts operational strategy from reactive mitigation to proactive optimisation[1]. As systems become more interconnected and instrumented at ever finer temporal granularity, the demand for models able to parse intricate temporal dynamics has driven a rapid evolution in forecasting methodology[2].

*This work is licensed under the Creative Commons Attribution 4.0 International License. Concurrently, all associated source code, probing scripts, and software artifacts generated throughout the project lifecycle are released under the open-source MIT license.

Copyright for any third-party material included in this work remains with its respective owners and must be respected accordingly.

During the preparation of this work, the authors used AI-based tools, including GPT-5.5, Gemini 3.6 Flash, Sonnet 5, and Opus 4.6, to support language refinement and improve clarity. All generated or suggested content was carefully reviewed, revised where necessary, and validated by the authors, who assume full responsibility for the final content of this work.

Evolution of Forecasting Methodologies

Historically, time-series analysis relied on classical statistical models such as AutoRegressive Integrated Moving Average (ARIMA) and Exponential Smoothing (ETS)[3, 1]. These methods are deeply rooted in mathematical theory, offering high interpretability, exact statistical guarantees, and robust performance on stable, low-dimensional datasets. Their reach, however, is bounded by the assumptions they rest upon, principally linearity and stationarity, and by the fact that each series is typically fitted in isolation.

To capture non-linear temporal behaviour without rigid feature engineering, the paradigm shifted towards deep learning. Recurrent Neural Networks (RNNs), and their gated variants such as Long Short-Term Memory (LSTM) networks[4], map sequences step by step through a recurrent hidden state, while dilated convolutional architectures reach comparable accuracy with a fixed receptive field and full parallelism over the sequence[5]. Trained jointly across many related series, these models established the global forecasting paradigm that underpins most contemporary production systems[6, 2].

More recently, the state of the art has been dominated by transformer-based architectures[7]. By replacing the recurrent bottleneck with self-attention, transformers process a sequence in parallel and compute dependencies between distant time steps explicitly, learning localised high-frequency fluctuations and long-range macro-patterns simultaneously[8]. Two costs follow from a naive application to sampled signals: attention scales quadratically with sequence length, and a single time step is a poor unit of meaning for a real-valued measurement. Both are addressed by *patching*, in which the series is segmented into contiguous sub-sequences of fixed length and each segment is embedded as one token, shortening the sequence by the patch factor while giving every token a locally semantic support[9].

The current stage of this evolution is represented by time-series foundation models, exemplified by the *Chronos* family of frameworks[10]. Chronos unifies time-series forecasting with the architectures powering modern natural language processing, treating time-series values not as continuous regression targets but as discrete language tokens, obtained by scaling and quantising real values into a fixed vocabulary. Its successor, Chronos-Bolt, replaces value quantisation with the patch-based scheme described above and a direct multi-quantile projection head, reducing inference cost by up to two orders of magnitude at equal or better accuracy[11, 12]. Patching is therefore not an incidental implementation detail, but the design choice that makes this class of models practical at scale.

Signal Representation and Classic Aliasing

Regardless of the downstream modelling architecture, the predictive fidelity of any forecasting pipeline remains strictly bounded by the digital representation of the underlying signal, and therefore by data acquisition, signal conditioning, and pre-processing.

The fundamental mathematical boundary governing the digitisation of continuous physical processes is the Nyquist–Shannon sampling theorem[13, 14]. The theorem states that a band-limited continuous-time signal can be perfectly captured and reconstructed without loss of information if and only if it is sampled uniformly at a sampling frequency f_s strictly greater than twice the highest frequency component $f_{\max}$ present within the signal:

$$f_s > 2f_{\max}. \quad (1)$$

The threshold $f_N = \frac{f_s}{2}$ is defined as the Nyquist frequency. When a system violates this criterion, either because of an insufficient sampling rate or because of the absence of a proper low-pass anti-aliasing filter, frequency components beyond the Nyquist limit fold back into the lower spectrum and become indistinguishable from genuine low-frequency content[15]. This distortive phenomenon, known as *aliasing*, introduces spurious, artificial low-frequency components that permanently mask and corrupt the true underlying dynamics of the signal. Crucially, no downstream model can undo it: the information is destroyed at acquisition time, not at inference time.

Classic aliasing is consequently treated as a solved engineering problem. Respecting (1) and filtering before sampling is customarily taken as sufficient evidence that a discrete sequence faithfully represents the

process that generated it, and that any frequency below f_N remains available to whatever model consumes the sequence. This report examines whether that sufficiency argument still holds once the sequence is consumed by a patch-based architecture.

Application Domain

To ground this abstract architectural evaluation, the study utilises a specific physical domain as an empirical reference layer. This reference setting models the solar energy produced by a photovoltaic array attached to an internal load, a local storage battery, and the national electricity grid[16]. The explicit operational objective within this system is to evaluate production variations over short-term future horizons, so that the system can dynamically modify the internal load and attach or detach the battery unit, with the aim of maintaining internal grid stability and enabling precise anomaly detection.

Within this infrastructure, a control AI agent is responsible for executing real-time tasks: dynamic load balancing, battery dispatch, state monitoring, active anomaly detection, and internal grid stability maintenance. section formalises the concepts, states, and relations of this setting as an OWL2 ontology, making the link between signal structure, learned representation, and domain meaning explicit.

This analysis is directly intended for researchers, engineers, and practitioners utilising foundation models within the Chronos family for time-series analysis, representation learning, and forecasting, as well as for individuals working in the field of energy management who are interested in characterising the fundamental limitations, structural boundaries, and latent capabilities of these architectures.

Purpose and Scope

This project formalises and empirically validates the phenomenon of *structural aliasing* in patch-based time-series foundation models, using Amazon Chronos-Bolt-Tiny as the target architecture. Chronos-Bolt-Tiny operates with a fixed patch size of $P = 16$, a stride of $S = 16$, and a context window of $T = 2048$ samples as default settings; its architecture comprises four encoder blocks followed by four decoder blocks, terminating in a direct quantile projection head[17, 12].

The study aims to provide a comprehensive understanding of the relationships between raw signal characteristics and the latent representations learned by the model, systematically identifying the critical links between sampling frequency (f_s), patch size (P), and stride (S), in order to propose actionable solutions to mitigate the phenomenon. The core objective is to challenge the baseline assumption that pre-trained temporal foundation models inherently preserve frequency attributes when processing Nyquist-compliant signals[18].

Patch-Based Tokenization: a Formal Description

Let $\mathbf{x} = \{x[n]\}_{n \in \mathbb{Z}}$ be a discrete-time signal representing the input time series. We define the patch-based tokenization operator $\mathcal{T}_{P,S}$ with patch length $P \in \mathbb{N}^+$ and stride $S \in \mathbb{N}^+$ as a mapping that transforms the signal $\mathbf{x}$ into a sequence of vector-valued tokens $\mathbf{T} = \{\mathbf{t}_k\}_{k \in \mathbb{Z}}$, where each token $\mathbf{t}_k \in \mathbb{R}^P$.

The k -th token is defined as

$$\mathbf{t}_k = \mathcal{T}_{P,S}(\mathbf{x})[k] = \begin{bmatrix} x[kS] \\ x[kS + 1] \\ \vdots \\ x[kS + P - 1] \end{bmatrix}. \quad (2)$$

Using element-wise indexing notation,

$$\mathbf{t}_k[m] = x[kS + m], \quad m \in \{0, \dots, P - 1\}. \quad (3)$$

Phase-Locking and Patch Indistinguishability

Consider a continuous-time periodic signal sampled uniformly at sampling frequency f_s , yielding the discrete-time sequence $x[n]$. We isolate a fundamental harmonic component and model it as

$$x[n] = \sin\left(2\pi \frac{f}{f_s} n + \phi\right), \quad (4)$$

where f denotes the signal frequency satisfying the Nyquist criterion $f < \frac{f_s}{2}$, and $\phi \in [0, 2\pi)$ is an arbitrary phase offset.

Phase-locking occurs when the structural phase profile of the signal repeats exactly under a translational shift equal to the stride S . Evaluating a stride translation yields

$$x[n + S] = \sin\left(2\pi \frac{f}{f_s} (n + S) + \phi\right) = \sin\left(2\pi \frac{f}{f_s} n + 2\pi \frac{fS}{f_s} + \phi\right). \quad (5)$$

For exact alignment between the signal and the tokenization operator, the phase accumulation over one stride must correspond to an integer number of 2π -periods

$$2\pi \frac{fS}{f_s} = 2\pi c, \quad c \in \mathbb{N}^+. \quad (6)$$

As a result, the signal exhibits exact stride-period translational invariance

$$x[n + S] = \sin\left(2\pi \frac{f}{f_s} n + 2\pi c + \phi\right) = \sin\left(2\pi \frac{f}{f_s} n + \phi\right) = x[n]. \quad (7)$$

The same derivation can be applied taking into account translational shifts equal to the patch size P , yielding a similar phase-locking condition

$$2\pi \frac{fP}{f_s} = 2\pi k \Rightarrow x[n + P] = x[n], \quad k \in \mathbb{N}^+. \quad (8)$$

Solving Equation 6 and Equation 8 for f yields the phase-locking frequency class $\mathcal{F}_{\text{lock}}$, which is defined as the set of frequencies producing self-repetition over one stride or over one patch.

$$\mathcal{F}_{\text{lock}} = \left\{ f \in \mathbb{R}^+ \mid f = c \frac{f_s}{S} \vee f = k \frac{f_s}{P}, \quad c, k \in \mathbb{N}^+, \quad f < \frac{f_s}{2} \right\}. \quad (9)$$

It is convenient to express both conditions in units that do not depend on the sampling rate. Introducing the cycles-per-stride and cycles-per-patch ratios

$$\text{cps} = \frac{fS}{f_s}, \quad \text{cpp} = \frac{fP}{f_s}, \quad (10)$$

Equation 6 and Equation 8 read simply $\text{cps} \in \mathbb{N}^+$ and $\text{cpp} \in \mathbb{N}^+$: $\mathcal{F}_{\text{lock}}$ is the set of frequencies completing a whole number of cycles within one stride or within one patch. This normalisation makes the two branches directly comparable across geometries and across sampling rates.

The collapse of consecutive tokens is not restricted to sinusoidal signals. Consider an arbitrary discrete-time signal satisfying the translational invariance condition

$$x[n + S] = x[n], \quad \forall n. \quad (11)$$

Taking into account two consecutive tokens, using the element-wise notation from Equation 3 and applying Equation 11 gives

$$\mathbf{t}_{k+1}[m] = x[(k+1)S + m] = x[kS + m + S] = x[kS + m] = \mathbf{t}_k[m]. \quad (12)$$

Since this equality holds for every component,

$$\mathbf{t}_{k+1} = \mathbf{t}_k. \quad (13)$$

The derivation above shows that some translations leave a signal unchanged; this repetition is not, by itself, evidence that two distinct signals become indistinguishable to the model. Structural aliasing is therefore treated as an emergent, falsifiable property of the trained representations[18]. At a phase-lock consecutive patch tokens coincide, $\mathbf{t}_k = \mathbf{t}_{k+1}$, so distinct inputs can share a token sequence. We diagnose the phenomenon by *readability* rather than by a fixed distance between representations: structural aliasing is present when the signal’s frequency can no longer be recovered from the model’s internal state at a locked frequency as well as it can at neighbouring, non-locked ones.

Analysis of Structural Degeneracies and Invariances

Equation 9 is the union of two distinct families,

$$\mathcal{F}_S = \left\{ c \frac{f_s}{S} \right\}_{c \in \mathbb{N}^+}, \quad \mathcal{F}_P = \left\{ k \frac{f_s}{P} \right\}_{k \in \mathbb{N}^+}, \quad \mathcal{F}_{\text{lock}} = (\mathcal{F}_S \cup \mathcal{F}_P) \cap \left[0, \frac{f_s}{2} \right), \quad (14)$$

which coincide if and only if $S = P$. The two families act on the token sequence in structurally different ways, and it is this difference, rather than the locking condition alone, that determines the behaviour of $\mathcal{T}_{P,S}$.

Stride branch: inter-patch degeneracy For $f \in \mathcal{F}_S$ the signal satisfies Equation 11, and Equation 13 applies directly: every token equals its predecessor, so the token sequence collapses into a static series of identical vectors. Two signals that agree on a single window of length P and share the stride period are then mapped to the same sequence, so $\mathcal{T}_{P,S}$ is not injective on this class. The stride branch therefore produces a genuine *degeneracy*: distinct inputs collide, and all inter-token temporal variation is eliminated.

Patch branch: token-space invariance For $f \in \mathcal{F}_P$ the signal period $T_0 = f_s/f$ divides P , so x is P -periodic and every token spans an integer number of complete cycles. Using Equation 3,

$$\mathbf{t}_k[m] = x[kS + m] = x[(kS + m) \bmod P] = \mathbf{t}_0[(kS + m) \bmod P], \quad (15)$$

so that

$$\mathbf{t}_k = \mathbf{C}^{kS \bmod P} \mathbf{t}_0, \quad (16)$$

where $\mathbf{C}$ denotes the circular left-shift operator on $\mathbb{R}^P$, $(\mathbf{C}\mathbf{v})[m] = \mathbf{v}[(m+1) \bmod P]$. Every token is a circular shift of the first, and the token alphabet is finite, containing at most $P/\gcd(P, S)$ distinct elements irrespective of the length of the context window. This is an *invariance* rather than a collapse: consecutive tokens need not be identical, but the whole sequence is generated by a single vector under a group action, and carries no information about the signal beyond $\mathbf{t}_0$ and the token index.

Case 1: Equal Patch Size and Stride ($P = S$) When patches are contiguous and non-overlapping the two branches coincide, $\mathcal{F}_S = \mathcal{F}_P$, and $kS \bmod P = 0$ for every k . The circular shift of Equation 16 reduces to the identity, and the invariance of the patch branch degenerates into the collapse of the stride branch,

$$\mathbf{t}_k = \mathbf{t}_0, \quad \forall k. \quad (17)$$

This is the worst case of the family: it is the only geometry in which *every* locked frequency produces exact and total collapse, with no residual structure in the token sequence. It is also the default configuration of the target architecture, which operates at $P = S = 16$.

Case 2: Overlapping Windows ($S < P$) When the stride is smaller than the patch length the two branches separate. Since $f_s/S > f_s/P$, the stride comb is the sparser of the two: strictly below Nyquist it contributes $\lceil S/2 \rceil - 1$ members against the $\lceil P/2 \rceil - 1$ of the patch comb, and whenever S divides P the stride comb is a subset of the patch comb. Most locked frequencies in this regime therefore belong to $\mathcal{F}_P \setminus \mathcal{F}_S$ and produce the shift-invariance of Equation 16, not total collapse.

Overlap does not weaken the exact condition. The derivation of Equation 13 makes no use of the relation between P and S , so at $f \in \mathcal{F}_S$ consecutive tokens are still exactly equal. What overlap changes is the behaviour *around* a lock: adjacent windows share $P - S$ samples by construction, so two consecutive tokens are constrained to agree on their common support regardless of frequency, and the transition into collapse is correspondingly smoother. Reducing the stride at fixed patch size therefore raises the lowest member of $\mathcal{F}_S$, thins the stride comb below Nyquist, and softens the local shape of the effect without removing it.

Case 3: Disjoint Windows ($S > P$) When the stride exceeds the patch length the windows leave $S - P$ unobserved samples between consecutive patches: the operator observes only the intervals $[kS, kS + P - 1]$, while the regions $[kS + P, (k + 1)S - 1]$ are never seen. Two signals that coincide on every observed window but differ on the gaps are mapped to the same token sequence, so $\mathcal{T}_{P,S}$ is non-injective for *any* f , independently of the locking condition. The resulting loss of information cannot be attributed to the interaction between signal periodicity and the tokenization grid, since it is already implied by the discarded samples. Configurations with $S > P$ are therefore excluded by construction from the experimental design, so that any observed degradation remains attributable to tokenization geometry rather than to missing data.

Consequences for observability Across all three cases the effect on the token sequence is the same in kind: at $f \in \mathcal{F}_{\text{lock}}$ the operator $\mathcal{T}_{P,S}$ ceases to be injective, and the frequency f can no longer be identified from the tokens alone. Under the stride branch the sequence carries a single vector; under the patch branch it carries a single vector and an index. In both cases the quantity that the downstream model would need in order to recover f has been removed before the first attention layer, and no property of the trained weights can restore it. Whether that formal loss actually translates into a measurable loss of *readability* in the learned representations is an empirical question, and it is the object of the hypotheses formulated in section .

Domain Description and Ontology Representation

The considered domain takes into account the energy generated by an asset infrastructure centered around a photovoltaic array (`pv:PhotovoltaicArray`), operating within a localized smart micro-grid ecosystem. This environment is dynamically monitored, analyzed, and managed by an autonomous AI control agent (`pv:ControlAgent`) that balances power flows and detects system failures.

Physical Assets and Grid Topology

The physical core of the system is composed of four primary entities under the physical component (`pv:PhysicalComponent`) class, which are physically connected to (`pv:connectedTo`) one another to form the micro-grid’s electrical topology:

- The photovoltaic array serves as the primary generation source, defined by its physical size (`pv:panelCount`) and inverter nameplate capacity (`pv:nominalPowerW`).
- The array directly supplies power to (`pv:suppliesTo`) three downstream assets: the external national electricity grid (`pv:NationalGrid`) interface (an infinite bidirectional bus for energy exchange), local internal consumption loads (`pv:InternalLoad`) representing dynamic demand, and a local electrochemical storage unit (`pv:BatteryStorage`).
- Functionally, the battery storage unit actively buffers (`pv:buffers`) the photovoltaic array, absorbing excess generation or discharging power during deficits.

Telemetry Streams and Tokenization Layer

To evaluate system performance, data streams are continuously collected across two operational vectors:

- The photovoltaic array measures (`pv:measures`) a continuous time-series signal (`pv:Signal`). This stream encapsulates instantaneous data properties such as photovoltaic power (`pv:pvPowerW`), global horizontal irradiance (`pv:horizontalIrradianceWm2`), and tilted surface irradiance (`pv:tiltedIrradianceWm2`), alongside frequency-domain metadata including the raw sampling rate (`pv:samplingFrequencyHz`), dominant periodic cycle frequency (`pv:fundamentalFrequencyHz`), wave amplitude (`pv:amplitudeW`), and phase offset relative to epoch (`pv:phaseRad`).
- Concurrently, the array is mapped via an environmental snapshot relationship (`pv:hasWeatherObservation`) to a weather observation log (`pv:WeatherObservation`), which tracks atmospheric variables like ambient air temperature (`pv:airTemperatureC`), wind speed (`pv:windSpeedMs`), and wind direction (`pv:windDirectionDeg`).

To prepare this continuous telemetry for deep learning inference, the raw signal passes through a processing layer that tokenizes (`pv:tokenizes`) it into a patched signal representation (`pv:PatchedSignal`). This sequence chunking strategy, tailored for foundation architectures like Chronos, is explicitly governed by architectural attributes: the token block size (`pv:patchSize`), window step stride (`pv:stride`), and calculated overlap ratio (`pv:overlapRatio`).

Inferred Environmental Contexts and System Anomalies

An execution reasoning engine dynamically updates the operational metadata of the system based on physical and mathematical constraints. The system extracts a baseline generation state (`pv:GenerationState`) derived from the tilted surface irradiance feature, classifying the current solar environment into one of four mutually exclusive subclasses: full peak production clear sky (`pv:ClearSkyState`), intermittent cloud cover (`pv:PartialCloudState`), dense cloud cover overcast (`pv:OvercastState`), or zero usable irradiance nighttime (`pv:NighttimeState`).

Simultaneously, the reasoner assesses system health by validating whether the photovoltaic array exhibits an anomaly (`pv:hasAnomaly`). It detects six distinct anomaly states (`pv:AnomalyState`):

- **Clipping Anomaly (`pv:ClippingAnomaly`):** Occurs when solar power saturates at or above inverter nominal capacity during periods of high peak irradiance.
- **Inverter Trip Anomaly (`pv:InverterTripAnomaly`):** A hard breaker or component disconnection where output drops to zero despite significant solar resource availability.
- **Ramp Anomaly (`pv:RampAnomaly`):** Identified when the dynamic rate of power change (`pv:deltaPowerW`) between consecutive intervals exceeds safe structural ramp thresholds.
- **Soiling Anomaly (`pv:SoilingAnomaly`):** A persistent drop in normalized conversion efficiency, indicating panel surface contamination such as dust or soil accumulation.
- **Overheating Anomaly (`pv:OverheatingAnomaly`):** High ambient heat conditions inducing thermal derating, which depresses panel production efficiency under strong irradiance.
- **Malfunction Anomaly (`pv:MalfunctionAnomaly`):** A structural hardware fault where generation drops to a severely degraded, near-zero baseline without clear environmental causes.

Machine Learning Vulnerabilities: Aliasing Events

In parallel with physical faults, the semantic layer evaluates computational data-processing vulnerabilities. If tokenization window choices conflict with the signal's cyclical boundaries, the patched signal exhibits aliasing (`pv:exhibitsAliasing`), indicating an ongoing aliasing event (`pv:AliasingEvent`) that compromises downstream forecasting model precision:

- A frequency blind spot (`pv:FrequencyBlindSpot`) is flagged when the continuous signal component matches an integer harmonic of the patch-induced frequency (`pv:patchInducedFrequencyHz`), rendering major cyclical patterns structurally invisible to the model (Hypotheses H1 and H1b).
- A phase aliasing effect (`pv:PhaseAliasingEffect`, Hypothesis H2) occurs when an active phase offset or spatial shift (`pv:phaseOffsetRad`) misaligns with token boundaries, degrading representation fidelity.
- An overlap distortion effect (`pv:OverlapEffect`, Hypothesis H3) reveals configuration setups where a low overlap ratio indicates suboptimal stride lengths, generating context sequence block redundancy.

Closed-Loop Autonomous Action

To restore grid equilibrium and preserve physical integrity, the photovoltaic array triggers actions (`pv:triggersAction`) belonging to the agent action (`pv:AgentAction`) class hierarchy. The autonomous control agent manages the closed-loop optimization loop and performs (`pv:performs`) four specialized mitigation interventions:

- It coordinates a battery dispatch intervention (`pv:BatteryDispatchAction`) to buffer grid fluctuations under volatile weather or to substitute lost generation during unexpected inverter breaker trips.
- It deploys a generation curtailment action (`pv:CurtailGenerationAction`) to throttle active power spikes during inverter clipping or critical panel overheating states.
- It initiates demand side load adjustment changes (`pv:LoadAdjustmentAction`) to shift internal consumer consumption steps during low overcast or zero production nighttime states.
- It issues a rapid warning alert (`pv:RampAlertAction`) to stability systems the instant an active ramp rate anomaly is detected.

Structural Aliasing Expected Impact

To ground the experimental validation within the semantic framework established by the system ontology, we map the digital signal processing phenomena directly to the ontological layer. Let a continuous-time physical signal (such as `pv:pvPowerW` or `pv:tiltedIrradianceWm2`) be sampled uniformly at a rate f_s (`pv:samplingFrequencyHz`) and transformed into a tokenized sequence via a `pv:PatchedSignal` operator defined by patch size P (`pv:patchSize`) and translation stride S (`pv:stride`).

The architectural grid frequency f_p (`pv:patchInducedFrequencyHz`) governs the macro-temporal resolution of the token space and is defined as:

$$f_p = \frac{f_s}{S} \quad (18)$$

Based on this parametric environment, we define three distinct, falsifiable hypotheses to characterize the boundaries of structural aliasing.

Hypothesis 1 (H_1): Frequency-Dependent Blind Spots

- **Ontological Alignment:** `pv:FrequencyBlindSpot` (asserted via SWRL inference rules R18 and R19).
- **Statement:** When the fundamental frequency f (`pv:fundamentalFrequencyHz`) of an input time-series signal matches an integer multiple of the patch-induced frequency f_p , the tokenization layer maps distinct cyclical variations into static, identical latent representations, creating discrete informational blind spots across the spectrum.
- **Mathematical Boundary:** An aliasing event is triggered when the frequency ratio satisfies the harmonic condition:

$$f = n \cdot f_p = n \left(\frac{f_s}{S} \right), \quad n \in \mathbb{N}^+ \quad (19)$$

Under this boundary condition, the column rank of the local trajectory matrix collapses: $\text{rank}(\mathbf{M}) = 1$ under full integer phase-locking, and $\text{rank}(\mathbf{M}) \leq q$ under rational phase-locking with denominator q (see Section).

- **Falsifiability Criterion:** This hypothesis is falsified if empirical probing models or closed-loop reconstruction tasks demonstrate uniform representation fidelity across the operational band $[2, \frac{f_s}{2}]$ Hz, without exhibiting statistically significant localized performance degradation or info-loss anomalies at the exact integer harmonics of f_p .

Hypothesis 2 (H_2): Phase Alignment and Temporal Displacement

- **Ontological Alignment:** `pv:PhaseAliasingEffect` (asserted via SWRL inference rule R20).
- **Statement:** The magnitude of structural information loss at an aliased harmonic frequency is non-deterministic and varies as a function of the temporal alignment between the signal’s peak-trough dynamics and the rigid boundaries of the tokenization grid.
- **Mathematical Boundary:** For a fixed harmonic frequency $f \in \mathcal{F}_{\text{lock}}$ and phase offset $\phi \in [0, 2\pi)$ in the signal

$$x[n] = \sin\left(2\pi \frac{f}{f_s} n + \phi\right), \quad (20)$$

the phase offset ϕ (radians) displaces the signal by

$$\tau = \frac{\phi f_s}{2\pi f} \quad (\text{samples}) \quad (21)$$

relative to the patch grid origin. Structural misalignment occurs when $\tau \bmod S \neq 0$, equivalently when

$$\phi \notin \left\{ 2\pi k \cdot \frac{f S}{f_s} : k \in \mathbb{Z} \right\}. \quad (22)$$

- **Falsifiability Criterion:** This hypothesis is falsified if information-theoretic tracking metrics (such as the Space Saving metric SV) or empirical probing classification accuracies remain strictly invariant across all randomly generated phase displacements $\phi \sim \text{Uniform}(0, 2\pi)$ at a locked harmonic frequency, proving that boundary location exercises no control over latent feature degradation.

Hypothesis 3 (H_3): Geometric Configuration and Overlap Mechanics

- **Ontological Alignment:** `pv:OverlapEffect` (asserted via SWRL inference rule R21).
- **Statement:** Altering the geometric relationship between the patch window length and the translational step size shifts the failure mode of the token space, moving the representation from absolute vector degeneracy to high-dimensional contextual redundancy.
- **Mathematical Boundary:** The geometric interaction is dictated by the token overlap ratio *or* (`pv:overlapRatio`):

$$or = \frac{P - S}{P} \quad (23)$$

When $or = 0$ ($P = S$), the windowing is strictly contiguous and non-overlapping; satisfying Equation (2) forces consecutive tokens to be mathematically identical ($\mathbf{t}_j = \mathbf{t}_{j+1}$), wiping out temporal gradients. When $or \rightarrow 1$ ($S \ll P$), successive patches share massive historical context; although the stride-bound phase-locking condition $x[n + S] = x[n]$ still holds, the expanded patch dimension P forces tokens to capture intra-patch phase variations that break the absolute vector identity, shifting the error into an over-smoothed informational redundancy.

- **Falsifiability Criterion:** This hypothesis is falsified if varying the architectural parameters P and S while keeping a locked signal frequency f yields an identical trajectory matrix rank profile and an identical distribution of downstream transformer attention weights, proving that the geometric layout of the patch grid does not modulate the structural aliasing expression.

Signal Generation

In order to evaluate the impact of structural aliasing on the Chronos-Bolt framework and its forecasting capabilities, we conducted a series of experiments using synthetic time-series data.

To observe the predicted values generated by Chronos-Bolt, we use three types of signal generators: a single-harmonic sinusoidal generator, a modified TSMixup generator, and a KernelSynth generator. The last two are based on methods proposed for Chronos training.[10]

Synthetic Probing Setup

Single-Harmonic Sinusoidal Generator This generator produces a simple sinusoidal signal with a single frequency component. The generated signal can be expressed mathematically as:

$$x(t) = A \cdot \sin(2\pi ft + \phi) \quad (24)$$

The continuous signal is then sampled at a specified rate to create a discrete time-series. The parameters of the generator, such as amplitude, frequency, and phase, can be adjusted to create different sinusoidal signals for testing purposes.

Light TSMixup Generator We propose a controlled probing variant of the TSMixup method described in the Chronos data augmentation procedure.[10] The original TSMixup algorithm samples subsequences from real training datasets, applies mean scaling to each sampled time series, draws mixing coefficients from a symmetric Dirichlet distribution, and returns their convex combination.

Light TSMixup preserves this stochastic mixing structure, but replaces the real dataset collection with a controlled pool of sinusoidal signals. Each base signal is defined by frequency, amplitude, phase, length, and sampling frequency. This simplification makes the generated signals easier to interpret when testing whether Chronos-Bolt exhibits frequency-dependent blind spots or phase-locking effects.

The parameters of the generator, such as the maximum number of sinusoidal components, Dirichlet concentration, target length, and sampling frequency, can be adjusted to create different synthetic signals for testing purposes. The pseudocode for the Light TSMixup generator is listed in Appendix B.

KernelSynth Generator Since TSMixup is able to produce only simple composition of sinusoidal signals, we implement KernelSynth as a more complex alternative, which can generate signals with richer frequency content and temporal dynamics. By comparing the performance of Chronos-Bolt on signals generated by both methods, we can gain insights into how structural aliasing affects the model’s ability to learn and forecast time-series data with varying levels of complexity.

The KernelSynth generator is also described in the Chronos data augmentation procedure.[10] Its pseudocode and kernel bank are listed in Appendix C.

This generator produces synthetic signals by combining different kernel functions, using randomly selected weights from a normal distribution. The selected kernels are combined using sum and product operators.

As parameters, the generator takes the maximum number of kernels to combine, and the time series output length. The selected kernel functions, the pseudo-code of the method, and the parameters used are described in Appendix C.

Bayesian Analysis

Each hypothesis receives its own measurement, its own model and its own estimand. Table 2 lists the models with the value each claim predicts, Table 3 every prior, and Table 4 the rule that turns a posterior into a verdict. All are ordinary regressions; only the quantity on the left and the question the coefficient answers change between them.

Each model is built the same way: a measurement, a linear predictor μ_i that adds one offset per group the observation belongs to, and a likelihood. One coefficient is the estimand and the rest keep it unconfounded. Table 3 gives every prior with the role its parameter plays. The range over which the prior scale is varied as a sensitivity check is $\{0.25, 0.5, 1.0\}$; all are centred on no effect, so a posterior that has not moved from its prior is itself a report of an absent effect.

H1

The behavioural measurement is the forecast amplitude recovery $R = A_{\text{pred}}/A_{\text{true}}$, collected in matched triplets: each lock f_k with its two controls, all three sharing the same background realisation and the same phase, which is what makes the contrast paired. The contrast used is

$$d = \log(R_k + 0.01) - \frac{1}{2} [\log(R_- + 0.01) + \log(R_+ + 0.01)], \quad (25)$$

one number per triplet, negative when the lock recovers less than its neighbours. It can be formed only because the injected tone has a known frequency. Model A fits it as

$$d_i \sim \text{Student-}t_4(\mu_i, \sigma), \quad \mu_i = \beta_{c[i]} + u_{k[i]} + u_{b[i]}, \quad \beta_c \sim \mathcal{N}\left(\bar{\beta} + \delta_O \tilde{O}_c + \delta_P \widetilde{\log P_c}, \tau\right). \quad (26)$$

Equation 26 has four components, each motivated separately.

- **Three offsets in μ_i :** β_c per configuration, u_k per lock harmonic and u_b per background draw, so that neither an unusual frequency nor a single corpus can be mistaken for the effect.
- **Student- t_4 , not Gaussian.** Where the model rebuilds nothing, recovery is near zero and the logarithm returns very large contrasts; under a Gaussian a small number of them would determine the estimate.
- **β_c drawn around $\bar{\beta}$,** making the geometries instances of one phenomenon rather than unrelated experiments. Two consequences follow. $\bar{\beta}$ exists as a parameter, which H1 requires, being a claim about

Run	Patch (P)	Stride (S)	overlap	tokens	$S \mid P$	$P \mid S$
1	8	4	0.50	511	yes	no
2	8	8	0.00	256	yes	yes
3	16	4	0.75	509	yes	no
4	16	8	0.50	255	yes	no
5	16	12	0.25	170	no	no
6	16	16	0.00	128	yes	yes
7	24	8	0.67	254	yes	no
8	24	12	0.50	169	yes	no
9	24	16	0.33	127	no	no
10	24	20	0.17	102	no	no
11	24	24	0.00	85	yes	yes
12	32	8	0.75	253	yes	no
13	32	12	0.62	169	no	no
14	32	16	0.50	127	yes	no
15	32	20	0.38	101	no	no
16	32	24	0.25	85	no	no
17	32	32	0.00	64	yes	yes

Table 1: The extended configurations retained for the analysis. All are retrained from scratch under the same 100k-step budget and from the same seed. $S \mid P$ means every stride lock is also a patch null, and $P \mid S$ the converse; both hold only at $P = S$, where the two branches are the same set of frequencies.

Model	Claim	Estimand	Predicts	Input
A	H1 behavioural	$e^{\bar{\beta}}$: recovery at a lock / at its controls	< 1	contrasts
B	H1 representational	$e^{\theta_{lock}}$: codelength at a lock / elsewhere	> 1	mdl_cells
C	H2	σ_ϕ : spread of the deficit across phase	≈ 0	contrasts
D1	H3 location	θ_S, θ_P : dip depth on each grid, by LOO	both < 0	collapse
D2	H3a stride branch	κ_S : measured / stride-predicted spacing	$= 1$	sites
D2	H3b patch branch	κ_P : measured / patch-predicted spacing	$= 1$	sites
A'	M1 mitigation	δ_O : change in the deficit per unit of overlap	< 0	contrasts

Table 2: The models, the quantity whose posterior decides each claim, the value the claim predicts, and the probing table each is fitted to. H1 is tested twice, on the forecast (A) and on the representation (B), so that a discrepancy between them stays visible. H3 is split into its two branches, since $\mathcal{F}_{lock}$ is a union and a frequency may belong to either. M1 is not a separate model but the configuration level of A, promoted to a named estimand.

patch-based tokenisation and not about a single checkpoint; and each geometry contributes equal weight, so the largest design cannot dominate the estimate when the number of usable triplets varies several-fold with the in-band lock count.

- **Two configuration-level covariates**, estimable together rather than confounded because the design varies overlap and patch size independently. Overlap enters as a ratio, patch size as $\log P$ because the patch grid has spacing f_s/P and equal steps in $\log P$ are equal ratios of spacing. The tilde denotes centring on the design, $\tilde{O}_c = (O_c - \bar{O})/0.5$ and $\log \tilde{P}_c = \log P_c - \log \bar{P}$, which leaves $\bar{\beta}$ the effect at the average configuration rather than an extrapolation to $O = 0$ and $P = 1$.

Model B poses the same question of the representation, on a frequency-local prequential codelength: at each f_c a probe separates tones at $f_c \pm 1$ Hz from a captured internal state. Ten states are probed: the encoder [REG] token after each encoder block, the decoder state after each decoder block, the pre-projection vector and the quantile head; the decoder-side vectors are named decoder states because Chronos-Bolt carries [REG] in the encoder only. The regression is Gamma with a log link, so $e^{\theta_{lock}}$ is the factor by which the codelength of a locked frequency increases, with probe stage and geometry as zero-mean offsets since codelengths differ several-fold between stages. Deliverable 1’s band tasks summarise hundreds of frequencies at once, so the lock indicator has nothing to attach to; they are kept as a cross-check.

H2

H2 uses the same triplets with one change. Equation 25 has already averaged over phase, so the phase index is kept instead and the response becomes the per-phase deficit $y_i = \log(R_{ctrl,i} + 0.01) - \log(R_{lock,i} + 0.01)$, which is d with its sign reversed and positive when the lock is worse. Model C is Model A on that response, without the configuration-level covariates and with the phase circle cut into eight slices, each given its own offset u_p :

$$y_i \sim \text{Student-}t_4(\mu_i, \sigma), \quad \mu_i = \beta_{c[i]} + u_{k[i]} + u_{b[i]} + u_{p[i]}, \quad u_p \sim \mathcal{N}(0, \sigma_\phi). \quad (27)$$

The estimand σ_ϕ is the spread of those eight offsets, and it answers the question with one number. If the deficit depends on where in its cycle the signal starts, the offsets differ and σ_ϕ is large. If the geometry alone determines it, they are equal and σ_ϕ is near zero, as H2 predicts. Slices are used rather than a fitted curve because they assume nothing about the form a dependence might take. Dropping u_p gives a nested null, and comparing the two by leave-one-out cross-validation (LOO) separates the size of the phase effect from the evidence that it exists.

Model	Parameter	Prior	Role
A, C	$\bar{\beta}$	Student- $t_4(0, 0.5)$	population phase-lock effect, on the log-recovery scale
A	δ_O	Student- $t_4(0, 0.5)$	slope in $\widetilde{O}_c$, the centred overlap; one unit = 0.5 of O
A	δ_P	Student- $t_4(0, 0.5)$	slope in $\widetilde{\log P}_c$, the centred log patch size
A, C	$\tau, \sigma_k, \sigma_b, \sigma$	Half-Student- $t_4(0, 0.5)$	configuration, harmonic, background and residual scales
C	σ_ϕ	Half- $\mathcal{N}(0, 0.25)$	spread of the eight per-phase offsets
B	α_0	$\mathcal{N}(\log \bar{y}, 1)$	baseline log codelength
B	θ_{lock}	$\mathcal{N}(0, 0.5)$	log codelength expansion at a lock
B	k	Gamma(2, 0.1)	Gamma shape (dispersion)
B	$\sigma_{st}, \sigma_{geo}$	Half- $\mathcal{N}(0, 1)$	spread of the probe-stage and geometry offsets
D1	$\alpha_g, \theta_S, \theta_P$	$\mathcal{N}(0, 1)$	per-geometry off-grid level and the two grid labels
D1	σ	Half- $\mathcal{N}(0, 1)$	residual scale of $\log z_g(f)$
D2	κ_S, κ_P	$\mathcal{N}(0, 1)$	measured spacing per unit of predicted spacing
D2	σ_F	Half- $\mathcal{N}(0, 5)$ Hz	scatter beyond the sweep-resolution floor Δf

Table 3: Every parameter with its prior and its role. The effect priors are those of Deliverable 1, extended by δ_P , which the design of Table 1 makes estimable.

H3

H3 was stated as two claims, one per branch. Both are tested on the token collapse $z_g(f)$, the dispersion of the input-patch embeddings across patches, which is zero when consecutive patches are identical. All geometries share one sweep grid: a uniform 1 Hz sweep unioned with the predicted sites of every configuration swept. No geometry is therefore scored only where its own theory predicts a dip, and non-integer sites such as 42.6 Hz for $S=12$ are covered.

The first question is where the dips are located. Each swept frequency carries two labels, one per branch:

$$\log z_g(f) \sim \mathcal{N}(\alpha_g + \theta_S \cdot \text{OnStrideGrid} + \theta_P \cdot \text{OnPatchGrid}, \sigma). \quad (28)$$

The logarithm makes the labels multiplicative on the per-geometry off-grid level α_g . The two label coefficients θ_S and θ_P are the estimands, and *H3* predicts both negative, so the two-label fit should win the LOO comparison against each label alone and against neither. The two labels are not competing explanations of the same dips, since a frequency may carry either or both. The two coefficients separate, however, only where exactly one label applies, which for θ_S means the geometries with $S \nmid P$.

The second question is whether the dips move. The fit above remains compatible with dips that merely sit on a fixed grid, so each branch is checked against its own scaling law. Every detected dip is assigned to the branch that predicts it, and sites belonging to both are set aside. The fundamental $\hat{f}_{1,g}^F$ of branch F in geometry g is then compared with the spacing that branch predicts, $\Delta_g^S = f_s/S_g$ or $\Delta_g^P = f_s/P_g$:

$$\hat{f}_{1,g}^F \sim \mathcal{N}\left(\kappa_F \Delta_g^F, \sqrt{\sigma_F^2 + \Delta f^2}\right). \quad (29)$$

The slope κ_F is the estimand, measured spacing per unit of predicted spacing, so $\kappa_F = 1$ denotes a branch that tracks its own arithmetic. The scatter σ_F about it is given a floor Δf , the sweep step, since a spacing

Claim	Rule	Prior	Reads as
H1 supported	$\Pr(\bar{\beta} < \log 0.8 \mid D) \geq 0.95$	0.34	at least 20% attenuation at a lock
H1 refuted	$\Pr(\bar{\beta} < \log 1.1 \mid D) \geq 0.95$	0.14	any effect is smaller than 10%
H2 supported	$\Pr(\sigma_\phi < \log 1.1 \mid D) \geq 0.95$	0.30	phase moves the deficit by under 10%
H3 location	two-label fit wins LOO, $\theta_S, \theta_P < 0$	—	dips sit on both predicted grids
H3a, H3b	$\Pr(\kappa_F - 1 < 0.1 \mid D) \geq 0.95$	0.05	that branch tracks its own parameter
M1 supported	$\Pr(\delta_O < 0 \mid D) \geq 0.95$, overlap term wins LOO	0.50	overlap reduces the deficit

Table 4: Decision rules. **Prior** is the probability each rule already scored under the priors of Table 3: a posterior counts as evidence only in as much as it has moved away from that value. A claim is supported at 0.95, refuted at 0.05, and otherwise inconclusive.

read off a discrete grid is no more precise than one step; without the floor, sites landing exactly on the prediction drive σ_F to zero and the posterior degenerates.

H3a, the stride branch The prediction is $\kappa_S = 1$: the stride sites track f_s/S one for one, so a site that remains fixed while S changes refutes it. Identification needs a series that holds P fixed and varies S , which Table 1 supplies at $P=16, 24$ and 32 .

H3b, the patch branch The prediction is $\kappa_P = 1$ in the same way. Identification needs a series that holds S fixed and varies P , which Table 1 supplies at $S=8, 12$ and 16 .

Decisions and validation

All signals are sampled at $f_s = 512$ Hz. Pure sinusoids are swept from 2 to 250 Hz in one-hertz steps at several non-redundant starting phases, so that every tone is fully representable under the Nyquist-Shannon theorem and a loss observed at a predicted lock frequency cannot be attributed to undersampling. A single seed (42) controls all stochastic generation, ensuring full reproducibility.

The complete experimental grid is built before any data are collected: one row per combination of geometry (P, S) , lock frequency, background type, and phase. Models A and C include a per-background random offset u_b with scale σ_b ; this lets the posterior separate generator-specific variation from the aliasing effect itself, so that a recovery difference between Light TSMixup and KernelSynth backgrounds is attributed to the generator rather than inflating or deflating the estimate of $\bar{\beta}$. For each experimental condition, 200 background draws are archived alongside the probing results, so that the exact inputs behind every posterior can be reloaded for verification.

Every threshold in Table 4 is fixed before a posterior is inspected, and validation comes before interpretation. The analysis is implemented in a single reproducible notebook organised in five sequential parts, each corresponding to one ingredient of the Bayesian inference.

Notebook workflow

The Bayesian analysis is implemented in one reproducible notebook, `bayesian_analysis.ipynb`. Its role is not to introduce a new method after the fact, but to execute the hypotheses, measurements, priors and decision rules defined above in a fixed order. The notebook therefore has a short setup stage followed by five inferential parts: priors, observation, likelihood, posterior inference and checks. Each part writes checkpoints to disk, so that an interrupted run can resume without changing the statistical design.

Part 0 (Setup). The notebook first installs only missing dependencies, locates the repository, fixes the random seed at 42, and creates the checkpoint directories. The reportable run uses 2000 warm-up draws, 2000 posterior draws, 4 chains and `target_accept = 0.9`. The design constants are imported from `probe_lib.py`: $f_s = 512$ Hz, a 480-sample context, a 64-sample prediction horizon, the 2–250 Hz analysis band, the retrained (P, S) geometries in Table 1, and the lock set $\mathcal{F}_{lock}$.

Part 1 (Priors). Before any Chronos output is loaded, the notebook builds the design skeleton: one planned row per geometry, lock frequency, background draw, generator and phase. It then declares the priors in Table 3, the decision thresholds in Table 4, and the sensitivity ladder $\{0.25, 0.5, 1.0\}$. Prior predictive checks are run on this skeleton rather than on a toy design, so the check asks whether the declared priors are centred on no effect, allow effects large enough to falsify or support the hypotheses, and make the decision thresholds reachable from both sides. The prior specification is saved as both JSON and Parquet, making later drift between the notebook and the report detectable.

Part 2 (Observation). This is the only GPU-dependent part, because it is the only part that loads Chronos-Bolt checkpoints. The collection module `collect.py` runs the geometries and writes five tidy tables: `contrasts`, `mdl_cells`, `mdl_bandtasks`, `collapse` and `sites`. `contrasts` feeds Models A and C with matched lock/control recovery measurements and retains the phase index for H2. `mdl_cells` feeds Model B with frequency-local prequential codelengths. `mdl_bandtasks` is retained only as a descriptive cross-check. `collapse` feeds Model D1 with token-dispersion profiles over the union frequency grid, and `sites` feeds Model D2 with detected branch-specific spacing estimates. Collection is sharded per geometry; a completed shard is reloaded rather than recomputed, and the generated TSMixup and KernelSynth backgrounds are archived so the observations behind a posterior can be inspected.

Part 3 (Likelihood). The notebook then defines the five PyMC model factories corresponding to Models A, B, C, D1 and D2. Each factory accepts a dataframe and returns an unsampled `pm.Model`; a shared sampling wrapper applies the same NUTS settings to every fit. Hierarchical effects use non-centred parameterisations where group scales can be small. Before fitting the real observations, the notebook performs parameter recovery for Model A: it simulates contrasts on the real design matrix at known values $\bar{\beta} \in \{0, \log 0.8, \log 0.5, \log 0.1\}$, refits the model, and checks that the posterior interval covers the injected truth and is narrower than the prior.

Part 4 (Posterior inference). The fitted posteriors are either sampled or reloaded from NetCDF checkpoints. For each hypothesis, the notebook reports the estimand on its natural scale, a 95% credible interval, the posterior probability of the preregistered decision threshold, and the practical-equivalence probability. Model comparisons use leave-one-out cross-validation where the report defined competing formulations: the phase-dependent and phase-free versions of Model C, the adjusted and literal versions of Model B, the configuration-level variants of Model A for mitigation, and the competing grid labels of Model D1.

Part 5 (Checks and verdicts). The final part tests whether the posterior can be trusted before interpreting it. Convergence diagnostics require $\hat{R} < 1.01$, adequate bulk and tail effective sample size, and zero divergences. Posterior predictive checks compare data generated from Model A against the observed contrasts, both pooled and stratified by configuration and generator. Prior sensitivity refits Model A across the scale ladder and with a Gaussian likelihood replacing the Student- t_4 likelihood; the H1 conclusion is reportable only if the decision probability and credible interval remain stable. LOO tables are read using the rule $|\text{elpd_diff}| < 2 \cdot \text{dse}$ as inconclusive. The notebook finally writes a verdict table with the three-way rule fixed before observation: supported, refuted or inconclusive.

References

- [1] Rob J. Hyndman and George Athanasopoulos. *Forecasting: Principles and Practice*. 3rd ed. Melbourne, Australia: OTexts, 2021. ISBN: 978-0-9875071-3-6. URL: <https://otexts.com/fpp3/> (visited on 08/19/2026).
- [2] Konstantinos Benidis et al. “Deep Learning for Time Series Forecasting: Tutorial and Literature Survey”. In: *ACM Computing Surveys* 55.6 (Dec. 31, 2022), pp. 1–36. DOI: 10.1145/3533382. URL: <https://dl.acm.org/doi/10.1145/3533382> (visited on 08/19/2026).

- [3] George E. P. Box et al. *Time Series Analysis: Forecasting and Control*. 5th ed. Wiley Series in Probability and Statistics. Hoboken, NJ: John Wiley & Sons, 2015. ISBN: 978-1-118-67502-1.
- [4] Sepp Hochreiter and Jürgen Schmidhuber. “Long Short-Term Memory”. In: *Neural Computation* 9.8 (Nov. 15, 1997), pp. 1735–1780. DOI: 10.1162/neco.1997.9.8.1735.
- [5] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. *An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling*. Apr. 19, 2018. DOI: 10.48550/arXiv.1803.01271. arXiv: 1803.01271 [cs.LG]. URL: <https://arxiv.org/abs/1803.01271> (visited on 08/19/2026). Pre-published.
- [6] David Salinas et al. “DeepAR: Probabilistic Forecasting with Autoregressive Recurrent Networks”. In: *International Journal of Forecasting* 36.3 (July 2020), pp. 1181–1191. DOI: 10.1016/j.ijforecast.2019.07.001.
- [7] Ashish Vaswani et al. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems 30 (NIPS 2017)*. Long Beach, CA: Curran Associates, Inc., 2017, pp. 5998–6008. arXiv: 1706.03762. URL: <https://arxiv.org/abs/1706.03762> (visited on 08/19/2026).
- [8] Qingsong Wen et al. “Transformers in Time Series: A Survey”. In: *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence (IJCAI-23)*. Macao, SAR: International Joint Conferences on Artificial Intelligence Organization, Aug. 2023, pp. 6778–6786. DOI: 10.24963/ijcai.2023/759. arXiv: 2202.07125.
- [9] Yuqi Nie et al. “A Time Series Is Worth 64 Words: Long-term Forecasting with Transformers”. In: *The Eleventh International Conference on Learning Representations (ICLR 2023)*. Kigali, Rwanda, 2023. arXiv: 2211.14730. URL: <https://openreview.net/forum?id=Jbdc0vT0col> (visited on 08/19/2026).
- [10] Abdul Fatir Ansari et al. *Chronos: Learning the Language of Time Series*. Nov. 4, 2024. DOI: 10.48550/arXiv.2403.07815. arXiv: 2403.07815 [cs.LG]. URL: <http://arxiv.org/abs/2403.07815> (visited on 05/13/2026). Pre-published.
- [11] Abdul Fatir Ansari et al. *Fast and Accurate Zero-Shot Forecasting with Chronos-Bolt and AutoGluon / Artificial Intelligence*. Dec. 2, 2024. URL: <https://aws.amazon.com/blogs/machine-learning/fast-and-accurate-zero-shot-forecasting-with-chronos-bolt-and-autogluon/> (visited on 05/13/2026).
- [12] *ChronosBoltPipeline | Amazon-Science/Chronos-Forecasting*. DeepWiki. URL: <https://deepwiki.com/amazon-science/chronos-forecasting/2.4-chronosboltpipeline> (visited on 05/13/2026).
- [13] Harry Nyquist. “Certain Topics in Telegraph Transmission Theory”. In: *Transactions of the American Institute of Electrical Engineers* 47.2 (Apr. 1928), pp. 617–644. DOI: 10.1109/T-AIEE.1928.5055024.
- [14] Claude E. Shannon. “Communication in the Presence of Noise”. In: *Proceedings of the IRE* 37.1 (Jan. 1949), pp. 10–21. DOI: 10.1109/JRPROC.1949.232969.
- [15] Alan V. Oppenheim and Ronald W. Schaffer. *Discrete-Time Signal Processing*. 3rd ed. Prentice-Hall Signal Processing Series. Upper Saddle River, NJ: Pearson, 2010. ISBN: 978-0-13-198842-2.
- [16] Sonia Leva et al. *Photovoltaic Power and Weather Parameters*. IEEE DataPort, Sept. 23, 2020. DOI: 10.21227/42V0-JZ14. URL: <https://ieee-dataport.org/open-access/photovoltaic-power-and-weather-parameters> (visited on 06/01/2026).
- [17] *Amazon/Chronos-Bolt-Tiny · Hugging Face*. Dec. 4, 2025. URL: <https://huggingface.co/amazon/chronos-bolt-tiny> (visited on 07/25/2026).
- [18] Alessandro Pagani et al. *Preliminary Insights in Chronos Frequency Data Understanding and Reconstruction*. May 7, 2026. DOI: 10.48550/arXiv.2605.06361. arXiv: 2605.06361 [cs.LG]. URL: <http://arxiv.org/abs/2605.06361> (visited on 05/13/2026). Pre-published.

Appendix A: Ontology Representation

The core ontology architecture bridges the gap between cyber-physical energy systems and deep learning signal processing domain knowledge. It categorizes the application setting into five main conceptual pillars rooted under the ontological hierarchy:

- **Physical Subsystem (pv:PhysicalComponent):** Represents the hardware infrastructure of the micro-grid, including the solar array, electrochemical storage, consumer loads, and the macro grid tie.
- **Cognitive Layer (pv:ControlAgent):** The artificial intelligence system responsible for autonomous grid optimization, load balancing, dispatch scheduling, and proactive vulnerability mitigation.
- **Signal Processing Layer (pv:Signal, pv:PatchedSignal):** Tracks the raw telemetry inputs (both time-domain features and frequency-domain components) along with their tokenized representation configurations (patch dimensions, stride values) tailored for the Chronos foundation architecture.
- **Environmental Tracking (pv:WeatherObservation):** Captures concurrent meteorological snapshots (air temperature, wind vectors) influencing structural physics and electrical efficiency.
- **Inferred Operational States:** Dynamic metadata classified into generation levels (pv:GenerationState), systemic disruptions (pv:AnomalyState), control directives (pv:AgentAction), and signal degradation hazards (pv:AliasingEvent).

Core Concept Taxonomy

The formal classes defined within the schema are explicitly detailed below:

- pv:PhotovoltaicArray** The primary renewable energy asset. Characterized by panel count and nameplate nominal capacity. Source of telemetry streams; associated with weather conditions and subject to environmental/electrical anomalies.
- pv:BatteryStorage** Local electrochemical reservoir capable of dynamic grid connection/isolation. Characterized by capacity and instantaneous State of Charge (SoC).
- pv:InternalLoad** Represents internal industrial or residential consumption demands. Subject to adjustable, demand-side management interventions.
- pv:NationalGrid** The external electrical macro-grid interface, acting as a bidirectional infinite bus for excess export or deficit imports.
- pv:Signal** Time-series measurement stream containing global horizontal irradiance (G_h), tilted surface irradiance (G_{tilt}), instantaneous power output (P_{pv}), and analytical frequency descriptors (sampling rate, fundamental frequency, amplitude, phase).
- pv:PatchedSignal** The Chronos-specific tokenized structure derived from a raw signal. Governed by a selected window chunking mechanism (*patchSize*, *stride*), establishing a structural *overlapRatio* and a downstream *patchInducedFrequency*.

Semantic Relations & Properties

Object Properties (Entity-to-Entity Relationships)

The physical, functional, and inferred linkages across concepts are formalized using the following object properties:

- $\langle \text{pv:PhotovoltaicArray} \rangle \text{ pv:measures } \langle \text{pv:Signal} \rangle$
- $\langle \text{pv:PhotovoltaicArray} \rangle \text{ pv:hasWeatherObservation } \langle \text{pv:WeatherObservation} \rangle$
- $\langle \text{pv:PhotovoltaicArray} \rangle \text{ pv:hasGenerationState } \langle \text{pv:GenerationState} \rangle$ (*Inferred via SWRL*)
- $\langle \text{pv:PhotovoltaicArray} \rangle \text{ pv:hasAnomaly } \langle \text{pv:AnomalyState} \rangle$ (*Inferred via SWRL*)
- $\langle \text{pv:PhotovoltaicArray} \rangle \text{ pv:triggersAction } \langle \text{pv:AgentAction} \rangle$ (*Inferred via SWRL*)

- $\langle \text{pv:Signal} \rangle \text{ pv:tokenizes } \langle \text{pv:PatchedSignal} \rangle$
- $\langle \text{pv:PatchedSignal} \rangle \text{ pv:exhibitsAliasing } \langle \text{pv:AliasingEvent} \rangle$ (*Inferred via SWRL*)
- $\langle \text{pv:BatteryStorage} \rangle \text{ pv:buffers } \langle \text{pv:PhotovoltaicArray} \rangle$
- $\langle \text{pv:PhysicalComponent} \rangle \text{ pv:connectedTo } \langle \text{pv:PhysicalComponent} \rangle$ (*Symmetric Property*)

Data Properties (Entity Attributes)

Key quantitative attributes defining system conditions include:

- **Electrical/Solar:** $\text{pv:pvPowerW } (P_{\text{pv}})$, $\text{pv:nominalPowerW } (P_{\text{nom}})$, $\text{pv:tiltedIrradianceWm2 } (G_{\text{tilt}})$, $\text{pv:deltaPowerW } (\Delta P)$.
- **Meteorological:** $\text{pv:airTemperatureC } (T_{\text{air}})$, $\text{pv:windSpeedMs } (W_s)$.
- **Tokenization Architecture:** $\text{pv:patchSize } (N_p)$, $\text{pv:stride } (S)$, $\text{pv:overlapRatio } (\gamma)$, $\text{pv:patchInducedFrequency } (f_p)$.
- **Signal Descriptors:** $\text{pv:samplingFrequencyHz } (f_s)$, $\text{pv:fundamentalFrequencyHz } (f_0)$, $\text{pv:phaseRad } (\phi)$.

Knowledge-Driven Inference System (SWRL Rules)

Real-time diagnostic, classification, and mitigation logic are automated using a rule-based engine. The standard logical form of these production rules is detailed below:

Environmental Context Extraction

$$\begin{aligned} \text{PhotovoltaicArray}(?pv) \wedge \text{measures}(?pv, ?s) \wedge \text{tiltedIrradianceWm2}(?s, ?g) \wedge (?g \geq 800.0) \\ \longrightarrow \text{hasGenerationState}(?pv, \text{ClearSky}) \end{aligned} \quad (30)$$

$$\begin{aligned} \text{PhotovoltaicArray}(?pv) \wedge \text{measures}(?pv, ?s) \wedge \text{tiltedIrradianceWm2}(?s, ?g) \\ \wedge (?g \geq 150.0) \wedge (?g < 800.0) \longrightarrow \text{hasGenerationState}(?pv, \text{PartialCloud}) \end{aligned} \quad (31)$$

$$\begin{aligned} \text{PhotovoltaicArray}(?pv) \wedge \text{measures}(?pv, ?s) \wedge \text{tiltedIrradianceWm2}(?s, ?g) \\ \wedge (?g > 0.0) \wedge (?g < 150.0) \longrightarrow \text{hasGenerationState}(?pv, \text{Overcast}) \end{aligned} \quad (32)$$

$$\begin{aligned} \text{PhotovoltaicArray}(?pv) \wedge \text{measures}(?pv, ?s) \wedge \text{tiltedIrradianceWm2}(?s, ?g) \wedge (?g \leq 0.0) \\ \longrightarrow \text{hasGenerationState}(?pv, \text{Nighttime}) \end{aligned} \quad (33)$$

Cyber-Physical Anomaly Diagnostics

- **Inverter Clipping Anomaly (R5):**

$$\begin{aligned} \text{PhotovoltaicArray}(?pv) \wedge \text{measures}(?pv, ?s) \wedge \text{pvPowerW}(?s, ?p) \wedge \text{nominalPowerW}(?pv, ?nom) \\ \wedge (?p \geq ?nom) \wedge \text{tiltedIrradianceWm2}(?s, ?g) \wedge (?g > 800.0) \\ \longrightarrow \text{hasAnomaly}(?pv, \text{Clipping}) \end{aligned} \quad (34)$$

- **Inverter Trip Anomaly (R6):**

$$\begin{aligned} \text{PhotovoltaicArray}(?pv) \wedge \text{measures}(?pv, ?s) \wedge \text{pvPowerW}(?s, ?p) \wedge (?p \leq 0.0) \\ \wedge \text{tiltedIrradianceWm2}(?s, ?g) \wedge (?g > 200.0) \longrightarrow \text{hasAnomaly}(?pv, \text{InverterTrip}) \end{aligned} \quad (35)$$

- **Ramp Event Anomaly (R7):**

$$\text{PhotovoltaicArray}(?pv) \wedge \text{measures}(?pv, ?s) \wedge \text{deltaPowerW}(?s, ?dp) \wedge (?dp > 500.0) \longrightarrow \text{hasAnomaly}(?pv, \text{Ramp}) \quad (36)$$

- **Soiling Performance Deficit Anomaly (R8):**

$$\begin{aligned} & \text{PhotovoltaicArray}(?pv) \wedge \text{measures}(?pv, ?s) \wedge \text{tiltedIrradianceWm2}(?s, ?g) \wedge (?g > 400.0) \\ & \wedge \text{pvPowerW}(?s, ?p) \wedge (?p > 100.0) \wedge \text{swrlb:divide}(?r, ?p, ?g) \wedge (?r < 0.3) \\ & \longrightarrow \text{hasAnomaly}(?pv, \text{Soiling}) \end{aligned} \quad (37)$$

- **Thermal Overheating Derating Anomaly (R9):**

$$\begin{aligned} & \text{PhotovoltaicArray}(?pv) \wedge \text{hasWeatherObservation}(?pv, ?w) \wedge \text{airTemperatureC}(?w, ?t) \wedge (?t > 35.0) \\ & \wedge \text{measures}(?pv, ?s) \wedge \text{tiltedIrradianceWm2}(?s, ?g) \wedge (?g > 600.0) \\ & \wedge \text{pvPowerW}(?s, ?p) \wedge \text{swrlb:divide}(?r, ?p, ?g) \wedge (?r < 0.4) \\ & \longrightarrow \text{hasAnomaly}(?pv, \text{Overheating}) \end{aligned} \quad (38)$$

Autonomous Closed-Loop Control Actuation

Based on inferred operating profiles, actions are dynamically mapped to handle grid stresses:

$$\text{pv:hasAnomaly}(?pv, \text{pv:Ramp}) \longrightarrow \text{pv:triggersAction}(?pv, \text{pv:RampAlert}) \quad (39)$$

$$\text{pv:hasAnomaly}(?pv, \text{pv:Clipping}) \vee \text{pv:hasAnomaly}(?pv, \text{pv:Overheating}) \quad (40)$$

$$\longrightarrow \text{pv:triggersAction}(?pv, \text{pv:CurtailGeneration}) \quad (41)$$

$$\text{pv:hasAnomaly}(?pv, \text{pv:InverterTrip}) \vee \text{pv:hasGenerationState}(?pv, \text{pv:PartialCloud}) \quad (42)$$

$$\longrightarrow \text{pv:triggersAction}(?pv, \text{pv:BatteryDispatch}) \quad (43)$$

$$\text{pv:hasGenerationState}(?pv, \text{pv:Overcast}) \vee \text{pv:hasGenerationState}(?pv, \text{pv:Nighttime}) \quad (44)$$

$$\longrightarrow \text{pv:triggersAction}(?pv, \text{pv:LoadAdjustment}) \quad (45)$$

Chronos Structural Aliasing Hypotheses

The key scientific contribution of this architecture lies in identifying information loss bounds during time-series patching. The model detects structural blind spots via frequency domain reasoning:

Hypothesis H1 & H1b: Patch-Induced Frequency Blind Spots

The patch tokenization process introduces an artificial operating frequency profile f_p , mathematically defined by the relationship:

$$f_p = \frac{f_s}{S} \quad (46)$$

Where f_s represents the raw Nyquist sampling rate and S denotes the token step stride.

The appropriate detection tolerance derives from the frequency resolution of the context window of length N_{context} samples:

$$\delta f \approx \frac{f_s}{N_{\text{context}}}. \quad (47)$$

For Chronos-Bolt default context lengths (512–2048 samples), this gives $\delta f \in [f_s/2048, f_s/512]$. The tolerance fraction $\varepsilon = \delta f / f_p = S / N_{\text{context}}$ must be validated empirically; whether $\pm 10\%$ is wider or narrower than this resolution depends on the specific S and N_{context} chosen.

- **Primary Blind Spot (H1):**

$$\begin{aligned} & \text{Signal}(?s) \wedge \text{tokenizes}(?s, ?ps) \wedge \text{fundamentalFrequencyHz}(?s, ?f_0) \wedge \text{patchInducedFrequencyHz}(?ps, ?f_p) \\ & \wedge \text{swrlb:divide}(?r, ?f_0, ?f_p) \wedge (?r \geq 1 - \varepsilon) \wedge (?r \leq 1 + \varepsilon) \\ & \longrightarrow \text{exhibitsAliasing}(?ps, \text{H1_BlindSpot}) \end{aligned} \quad (48)$$

- **Secondary Harmonic Blind Spot (H1b):**

$$\begin{aligned} & \text{Signal}(?s) \wedge \text{tokenizes}(?s, ?ps) \wedge \text{fundamentalFrequencyHz}(?s, ?f_0) \wedge \text{patchInducedFrequencyHz}(?ps, ?f_p) \\ & \wedge \text{swrlb:divide}(?r, ?f_0, ?f_p) \wedge (?r \geq 2 - \varepsilon) \wedge (?r \leq 2 + \varepsilon) \\ & \longrightarrow \text{exhibitsAliasing}(?ps, \text{H1b_BlindSpot}) \end{aligned} \quad (49)$$

Hypothesis H2: Phase Aliasing Effect

Vulnerabilities appear when temporal offsets misalign with chunk window anchors, corrupting early representation extraction features ($0 < \phi < S$):

$$\begin{aligned} & \text{Signal}(?s) \wedge \text{tokenizes}(?s, ?ps) \wedge \text{phaseRad}(?s, ?\phi) \wedge \text{stride}(?ps, ?S) \\ & \wedge (?\phi > 0.0) \wedge (? \phi < ?S) \longrightarrow \text{exhibitsAliasing}(?ps, \text{H2_Phase}) \end{aligned} \quad (50)$$

Hypothesis H3: Overlap Information Distortion

When the overlap configuration ratio ($\gamma = \frac{N_p - S}{N_p}$) drops below a critical information-theoretic bound ($\gamma < 0.5$), consecutive contextual patches display heavy text-token gaps, rendering trend forecasts highly unstable:

$$\text{PatchedSignal}(?ps) \wedge \text{overlapRatio}(?ps, ?or) \wedge (?or < 0.5) \longrightarrow \text{exhibitsAliasing}(?ps, \text{H3_Overlap}) \quad (51)$$

Appendix B: Light TSMixup Pseudocode

Algorithm 1 Light TSMixup: Controlled Sinusoidal Time Series Mixup

Require: Sinusoidal pool $\mathcal{P} = \{p_1, \dots, p_{N_p}\}$, where each p_n is defined by frequency f_n , amplitude A_n , phase ϕ_n , source length T_n , and sampling frequency f_s ; maximum components $K = 10$; symmetric Dirichlet concentration $\alpha = 1.5$; minimum and maximum augmented lengths ($l_{\min} = 128, l_{\max} = 2048$).

Ensure: An augmented time series.

```

1:  $k \sim U\{1, K\}$                                 ▷ number of sinusoidal components to mix
2:  $l \sim U\{l_{\min}, l_{\max}\}$                         ▷ length of the augmented time series
3: for  $i \leftarrow 1$  to  $k$  do
4:    $n \sim U\{1, N_p\}$                                 ▷ sample a sinusoidal source
5:    $x_{1:l}^{(i)} \leftarrow \text{SampleSegment}(p_n, l, f_s)$     ▷ generate or crop a length- $l$  segment
6:    $\tilde{x}_{1:l}^{(i)} \leftarrow x_{1:l}^{(i)} / \left( \frac{1}{l} \sum_{j=1}^l |x_j^{(i)}| \right)$     ▷ apply mean scaling
7: end for
8:  $[\lambda_1, \dots, \lambda_k] \sim \text{Dir}([\alpha_1 = \alpha, \dots, \alpha_k = \alpha])$     ▷ sample mixing weights
9: return  $\sum_{i=1}^k \lambda_i \tilde{x}_{1:l}^{(i)}$                 ▷ take a weighted combination of scaled signals

```

Appendix C: KernelSynth Pseudocode and Kernel Bank

Algorithm 2 KernelSynth: Synthetic Data Generation using Gaussian Processes

Require: Kernel bank $\mathcal{K}$ listed in Table 5; maximum kernels per time series $J = 5$; generated length $l_{\text{syn}} = 1024$; optional numerical jitter $\epsilon = 0$.

Ensure: A synthetic time series $x_{1:l_{\text{syn}}}$.

```

1:  $j \sim U\{1, J\}$  ▷ sample the number of kernels
2:  $\{\kappa_1(t, t'), \dots, \kappa_j(t, t')\} \stackrel{\text{i.i.d.}}{\sim} \mathcal{K}$  ▷ sample kernels with replacement
3:  $t \leftarrow \text{linspace}(0, 1, l_{\text{syn}})$ 
4:  $\kappa^*(t, t') \leftarrow \kappa_1(t, t')$  ▷ initialize the composite covariance
5: for  $i \leftarrow 2$  to  $j$  do
6:    $\star \sim U\{+, \times\}$  ▷ sample a binary composition operator
7:    $\kappa^*(t, t') \leftarrow \kappa^*(t, t') \star \kappa_i(t, t')$  ▷ compose kernels
8: end for
9: if  $\epsilon > 0$  then
10:    $\kappa^*(t, t') \leftarrow \kappa^*(t, t') + \epsilon I$ 
11: end if
12:  $x_{1:l_{\text{syn}}} \sim \mathcal{GP}(0, \kappa^*(t, t'))$  ▷ sample from the GP prior
13: return  $x_{1:l_{\text{syn}}}$ 

```

Table 5: Kernel bank $\mathcal{K}$ used by KernelSynth.

Kernel	Formula	Hyperparameters
Constant	$\kappa_{\text{Const}}(x, x') = C$	$C = 1$
White Noise	$\kappa_{\text{White}}(x, x') = \sigma_n \cdot \mathbf{1}_{x=x'}$	$\sigma_n \in \{0.1, 1\}$
Linear	$\kappa_{\text{Lin}}(x, x') = \sigma_0^2 + x \cdot x'$	$\sigma_0 \in \{0, 1, 10\}$
RBF	$\kappa_{\text{RBF}}(x, x') = \exp\left(-\frac{\ x-x'\ ^2}{2l^2}\right)$	$l \in \{0.1, 1, 10\}$
Rational Quadratic	$\kappa_{\text{RQ}}(x, x') = \left(1 + \frac{\ x-x'\ ^2}{2\alpha}\right)^{-\alpha}$	$\alpha \in \{0.1, 1, 10\}, l = 1$
Periodic	$\kappa_{\text{Per}}(x, x') = \exp\left(-2 \sin^2\left(\pi \frac{\ x-x'\ }{p/l_{\text{syn}}}\right)\right)$	$p \in \{24, 48, 96, 168, 336, 672, 7, 14, 30, 60, 365, 730, 4, 26, 52, 4, 6, 12, 4, 40, 10\}$
